

Tribe RISC-V CPU

About

Tribe is a RV32 RISC-V CPU model written in the CppHDL C++ dialect. The model runs both as a native C++ simulation and as generated SystemVerilog through Verilator. One Tribe core contains a small in-order pipeline, private instruction and data L1 caches, CSR/trap support, interrupt routing, and optional Sv32 MMU/TLBs. `TribeTest<CPU_CORES>` composes one or more cores around a shared coherent L2 cache, clock-domain crossings, and AXI4-style memory ports; the simulation and SoC wrappers add memories and devices around that cluster.

The base implementation targets 32-bit integer software. Build-time configuration in `tribe_cpu/Config.h` selects optional blocks such as `ENABLE_ZICSR`, `ENABLE_RV32IA`, `ENABLE_ISR`, and `ENABLE_MMU_TLB`. The L2 memory data width is selected with `L2_AXI_WIDTH`; common test targets use 64, 128, and 256 bits. Main memory starts at `memory_base_in`, has runtime size `memory_size_in`, and is split into four L2 memory/device regions. The last region is used as uncached IO/MMIO space.

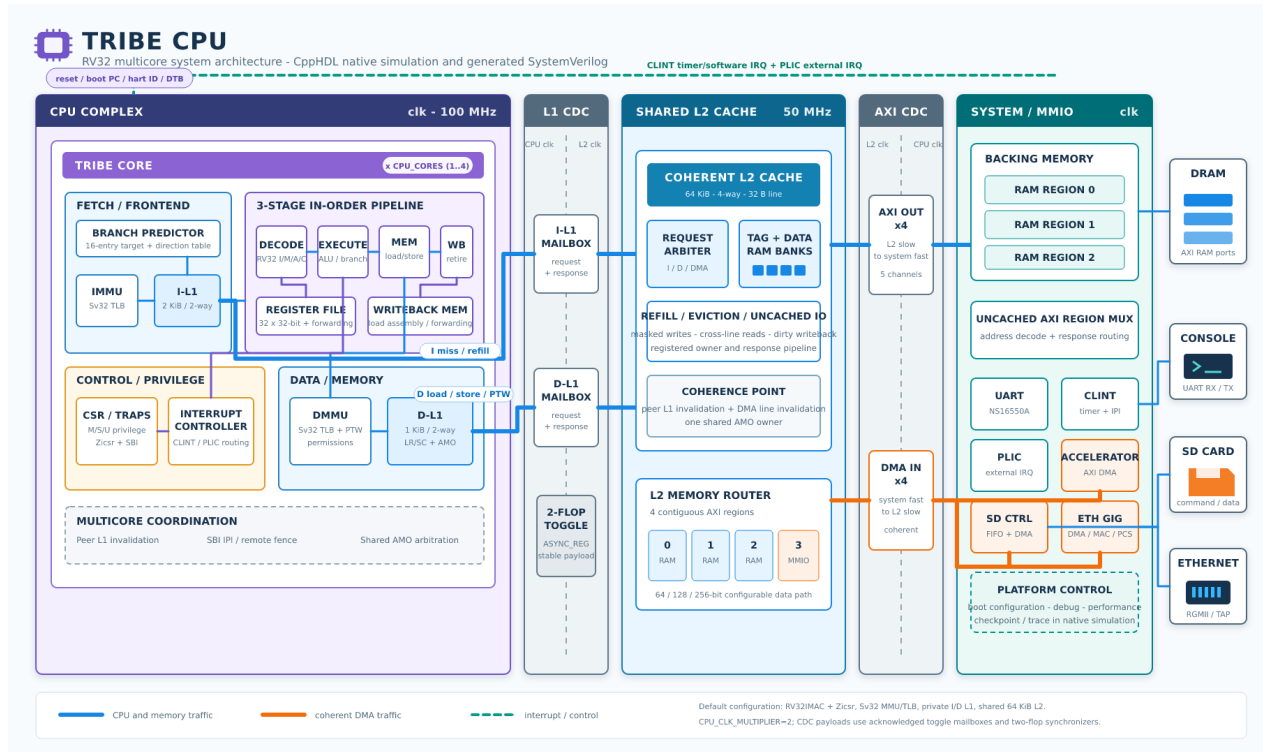


Figure 1: Tribe CPU architecture

Structure

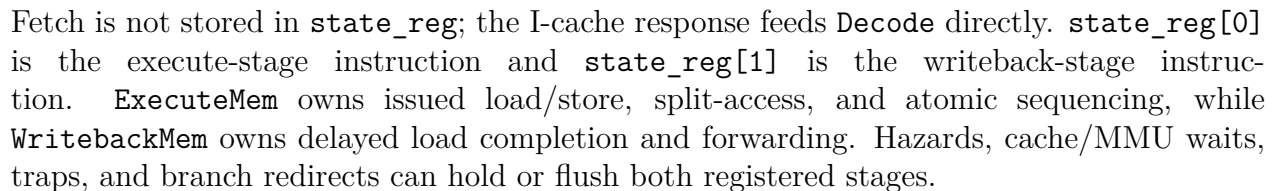
The single-core CPU is the `Tribe` module in `tribe_cpu/Tribe.h`. It instantiates these blocks in the primary `clk` domain:

Block	Source	Role
Decode	<code>tribe_cpu/Decode.h</code>	Instruction decode, register source selection, and <code>State</code> construction.
Execute	<code>tribe_cpu/Execute.h</code>	ALU operation and branch resolution.
ExecuteMem	<code>tribe_cpu/ExecuteMem.h</code>	Memory request generation, split access handling, and optional atomics.
WritebackMem	<code>tribe_cpu/WritebackMem.h</code>	Load response capture, split-load assembly, and store-to-load forwarding.
Writeback	<code>tribe_cpu/Writeback.h</code>	Architectural register writeback formatting.
CSR	<code>tribe_cpu/CSR.h</code>	CSR, privilege, trap, and return-from-trap state.
MMU_TLB	<code>tribe_cpu/MMU_TLB.h</code>	Optional Sv32 instruction/data address translation and page-table walking.
InterruptController	<code>tribe_cpu/InterruptController.h</code>	Optional CLINT and PLIC/external interrupt routing into CSR trap input.
File<32,32>	<code>File.h</code>	Integer register file.
L1Cache	<code>tribe_cpu/cache/L1Cache.h</code>	Separate instruction and data L1 caches.
BranchPredictor	<code>tribe_cpu/BranchPredictor.h</code>	Small direct-mapped branch predictor.

`TribeTest<CPU_CORES>` in `tribe_cpu/TribeTestModule.h` is the CPU-cluster composition point. It owns the core array, one instruction and one data `L1MemFastToSlowCdc` per core, the shared `L2Cache`, AXI CDCs on both sides of L2, peer-cache invalidation, SBI inter-hart routing, and multicore AMO arbitration.

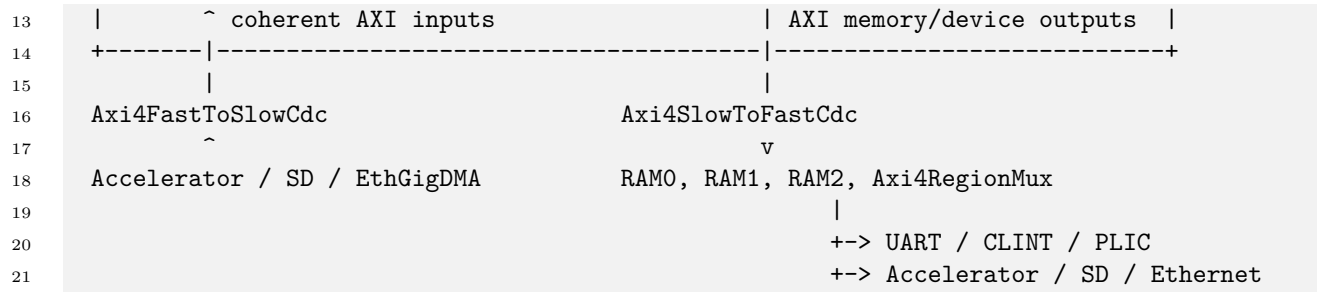
The executable simulation wrapper is `TestTribe` in `tribe_cpu/TribeTest.h`, driven by `tribe_cpu/main.cpp`. It instantiates three RAM regions, an IO region mux, NS16550A UART, CLINT, PLIC, Accelerator, SD controller plus SD verification card, and the ethgig DMA/MAC/PCS/PHY chain plus optional TAP-backed RGMII verification link.

CPU Pipeline



Principal connection of modules





The shared L2 is the coherence point. CPU private L1s use a level-held `L1MemIf`; coherent DMA masters enter L2 through `axi_in`; L2 reaches backing RAM and uncached MMIO through `axi_out`. The cluster uses explicit mailbox CDC modules at every fast/slow boundary. `System` instantiates a five-device IO mux and omits Ethernet; native `TestTribe` uses a six-device mux and includes the Ethernet chain.

Clocks

The design has two synchronous, phase-aligned clock inputs with distinct ownership:

Clock	Default frequency	Owned state
<code>clk</code>	100 MHz	CPU pipeline, register files, MMU/TLBs, private L1 caches, devices, RAM responders, region muxes, and the fast sides of all CDC modules.
<code>12_clock</code>	100 MHz / <code>CPU_CLK_MULTIPLIER</code>	Shared L2 tag/data RAM, L2 controller state, and the slow sides of all CDC modules.

`CPU_CLK_MULTIPLIER` is defined in `tribe_cpu/Config.h`, defaults to 2, must be positive, and must divide 100 MHz exactly. With the default, `12_clock` is 50 MHz and has a rising edge on every second primary-clock cycle. Native C++ and Verilator harnesses use the same unshifted edge relationship.

CppHDL conversion receives the clocks explicitly:

```

1 --primary_clock clk 100000000 \
2 --secondary_clock 12_clock 50000000

```

`Tribe`, its L1 caches, and all devices advance through `_work()/_strobe()` on `clk`. `L2Cache` advances only through `_work_12_clock()/_strobe_12_clock()`. `L1MemFastToSlowCdc`,

`Axi4FastToSlowCdc`, and `Axi4SlowToFastCdc` each contain state in both domains and transfer stable payloads with synchronized toggle handshakes. The two-flop toggle synchronizers carry `ASYNC_REG` attributes. Requests remain held until acknowledged, so correctness does not depend on a one-cycle pulse or on the current value of `CPU_CLK_MULTIPLIER`.

Reset is applied to both domains. Test harnesses keep reset active long enough for at least one `l2_clock` edge; otherwise fast-domain state could leave reset before L2 and the slow sides of the CDCs have sampled reset.

Module Inventory

The following tables cover the production `Module` classes, composition wrappers, and verification frontends under `tribe_cpu`. Small packed payload structs and enums are described with their owning module rather than listed as separate hardware modules.

Composition Modules

Module	Source	Purpose
<code>Tribe</code>	<code>Tribe.h</code>	One RV32 core, pipeline control, private I/D L1 caches, translation, interrupts, and L1 memory interfaces.
<code>TribeTest<CPU_CORES></code>	<code>TribeTestModule.h</code>	CPU cluster, shared L2, CDCs, peer invalidation, multicore SBI routing, and AMO ownership.
<code>System</code>	<code>SoC/System.cpp</code>	Synthesizable-style single-core SoC with internal RAM2 and five MMIO devices; exports RAM0/RAM1 AXI ports.
<code>SystemTest</code>	<code>SoC/System.cpp</code>	Native/Verilator host wrapper around <code>System</code> with external DRAM models and executable loading.

Module	Source	Purpose
TestTribe	TribeTest.h	Full simulation platform with memories, all devices, tracing, UART, checkpoints, performance counters, SD images, and TAP networking.

Core Modules

Module	Source	Purpose
Decode	Decode.h	Decodes RV32I/C/M/A/Zicsr instructions and builds the pipeline State .
Execute	Execute.h	Integer ALU, multiply/divide, comparisons, and branch target/decision logic.
ExecuteMem	ExecuteMem.h	Issues loads/stores, handles split accesses, and sequences LR/SC/AMO operations.
WritebackMem	WritebackMem.h	Matches cache responses, assembles split loads, and performs store-to-load forwarding.
Writeback	Writeback.h	Selects and formats the value written to the integer register file.
File	common/File.h	Two-read/two-write integer register file with same-cycle write forwarding and boot values for a0/a1 .
CSR	CSR.h	Machine/supervisor CSR state, privilege transitions, exception entry, and trap return.
InterruptController	InterruptController.h	Combines CLINT, PLIC, CSR enable, delegation, and privilege state into one interrupt request.

Module	Source	Purpose
MMU_TLB	MMU_TLB.h	Sv32 TLB, permission checks, direct physical window, and hardware page-table walker.
BranchPredictor	BranchPredictor.h	Direct-mapped target table with saturating direction counters.

Instruction semantics are supplied by `spec/Rv32i.h`, `Rv32ic.h`, `Rv32im.h`, `Rv32ia.h`, and `Zicsr.h`; `spec/State.h` defines the packed control record and operation enums passed through the pipeline.

Cache And CDC Modules

Module/layer	Source	Purpose
L1CacheGeometry	cache/l1/L1CacheGeometry.h	Shared line/set/tag/beat calculations and cacheability predicates.
L1CacheState	cache/l1/L1CacheState.h	L1 ports, RAMs, FSM/register state, and grouped request/response records.
L1CacheRequest	cache/l1/L1CacheRequest.h	Captures CPU requests and builds one grouped backing-memory request.
L1CacheRefill	cache/l1/L1CacheRefill.h	Accumulates refill beats and assembles cached or direct words.
L1CacheLookup	cache/l1/L1CacheLookup.h	Associative tag lookup, hit data selection, and RAM issue control.
L1CacheResponse	cache/l1/L1CacheResponse.h	Groups CPU response, busy, and performance outputs.
L1Cache	cache/l1/L1CacheController.h	Final L1 module that wires RAM ports and advances initialization, lookup, refill, and completion states.

Module/layer	Source	Purpose
L1MemIf	cache/L1MemIf.h	Level-held request/wait interface between an L1/MMU path and shared L2.
L1MemFastToSlowCdc	cache/L1MemCdc.h	One-entry request/response toggle mailbox from <code>clk</code> to <code>l2_clock</code> .
L2CacheGeometry	cache/l2/L2CacheGeometry.h	Shared L2 set/tag/line/beat geometry.
L2CacheByteOps	cache/l2/L2CacheByteOps.h	Masked and unaligned word merge/extract helpers.
L2CacheRegionRouter	cache/l2/L2CacheRegionRouter.h	Maps physical addresses to contiguous memory regions and uncached policy.
L2CacheTagOps	cache/l2/L2CacheTagOps.h	Packs, extracts, and searches valid/dirty/tag entries.
L2CacheResponseGlue	cache/l2/L2CacheResponseGlue.h	Slices cache lines into beats and joins cross-line reads.
L2CacheTimeoutOps	cache/l2/L2CacheTimeoutOps.h	Operation-age and timeout predicates used by diagnostics/tests.
L2CachePortOps	cache/l2/L2CachePortOps.h	Request-source priority and CPU wait helper policy.
L2CacheState	cache/l2/L2CacheState.h	L2 interfaces, physical RAMs, FSM, request/response records, and replacement state.
L2CacheRequest	cache/l2/L2CacheRequest.h	Arbitrates AXI, D, and I requests and captures a stable <code>CacheRequest</code> .
L2CacheMemory	cache/l2/L2CacheMemory.h	Routes refill, eviction, and uncached transactions to AXI memory/device ports.
L2CacheTagData	cache/l2/L2CacheTagData.h	Hit lookup, write merge, refill merge, eviction line, and read-beat datapaths.
L2CacheWait	cache/l2/L2CacheWait.h	Produces per-core I/D wait signals from registered response ownership.

Module/layer	Source	Purpose
L2Cache	cache/l2/L2CacheController.h	Final shared-cache controller and l2_clock state owner.
Axi4FastToSlowCdc	common/Axi4Cdc.h	Transfers independent AXI channels from clk masters to the L2 domain.
Axi4SlowToFastCdc	common/Axi4Cdc.h	Transfers L2 AXI masters to clk-domain RAM/device responders.

Interconnect, Storage, And Device Modules

Module	Source	Purpose
Axi4Ram	common/Axi4Ram.h	AXI responder backed by simulation/checkpointable memory.
Axi4RegionMux	common/Axi4RegionMux.h	Routes one uncached AXI region to multiple MMIO responders.
RAM	common/RAM.h	Single-port synchronous-read RAM used by cache tag/data storage.
Memory	common/Memory.h	Dual-port masked memory primitive with show-ahead or registered reads.
IOUART	devices/IOUART.h	Minimal write-only UART-style output device for bare-metal tests.
NS16550A	devices/NS16550A.h	16550-compatible UART subset with RX buffering and PLIC interrupt.
CLINT	devices/CLINT.h	Per-hart software interrupts, timer compare, and shared machine time.
PLIC	devices/PLIC.h	Priorities, pending/enables, thresholds, and claim/complete contexts.
Accelerator	devices/Accelerator.h	Test accelerator with local memory, PRBS generation, and coherent DMA.

Module	Source	Purpose
SDController	devices/sd/SDController.h	MMIO, PIO, descriptor/DMA, IRQ, and SD command/data control.
SDFifo	devices/sd/SDFifo.h	Small byte FIFO used by the SD datapath and tests.
SDPhysical	devices/sd/SDPhysical.h	Card-facing SD command/response byte-stream engine.
EthGigDMA	devices/net/ethgig/ethgigXilinx.h	AXI4-compatible Ethernet/DMA register and descriptor engine.
EthGigMAC	devices/net/ethgig/ethgigEthernet.h	Ethernet framing, padding, CRC, filtering, and payload streams.
EthGigPCS	devices/net/ethgig/ethgigBios.h	Discretionary buffering between MAC and PHY byte streams.
EthGigPHY	devices/net/ethgig/ethgigPHY.h	PHY nibble conversion and fixed-link MDIO register behavior.

common/Axi4.h defines the grouped `Axi4Driver`, `Axi4Responder`, and `Axi4If` channel types used by every AXI module.

Verification Modules

Module	Source	Purpose
SDCardVerif	verif/SDCardVerif.h	Host-side SD image/card behavior and checkpoint state.
SDCardVerifFrontend	verif/SDCardVerif.h	CppHDL module exposing the SD verification model as byte-stream ports.
RGMIIVerif	verif/RGMIIVerif.h	Host-side packet queues and RGMIi frame behavior.
RGMIIVerifFrontend	verif/RGMIIVerif.h	CppHDL module connecting packet verification to PHY nibble ports.

Module	Source	Purpose
EthGigTapSocket	TribeDebug.h	Native simulation Unix-socket endpoint for the host TAP bridge.

Main (Core)

Tribe is a small in-order CPU with a fetch/decode path followed by execute and writeback pipeline state. The state array tracks Decode-to-Execute and Execute-to-Writeback state; instruction fetch is driven directly from `pc`, I-cache output, branch prediction, and MMU translation. The `State` structure carries decoded instruction control fields through the core.

Top-level **Tribe** ports:

Port	Type	Description
<code>dmem*_out</code> , <code>imem_read_addr_out</code>	scalar debug ports	Current private-cache requests for trace and testbench use.
<code>reset_pc_in</code>	<code>uint32_t</code>	Reset PC.
<code>boot_hartid_in</code>	<code>uint32_t</code>	Reset value for <code>a0</code> , conventionally hart id.
<code>boot_dtb_addr_in</code>	<code>uint32_t</code>	Reset value for <code>a1</code> , conventionally device-tree address.
<code>boot_priv_in</code>	<code>u<2></code>	Initial privilege mode when CSR support is enabled.
<code>external_cache_invalidate_req</code>	<code>bool</code>	External request to invalidate L1 caches after coherent DMA writes when a wrapper needs software-visible freshness.
<code>peer_cache_invalidate*_in</code>	scalar ports	Multicore targeted private-cache and LR/SC reservation invalidation.
<code>memory_base_in</code>	<code>uint32_t</code>	Physical base address of the attached memory map.
<code>memory_size_in</code> , <code>mem_region_size_in</code>	map configuration	Lets the core identify translated RAM and direct/uncached device space.
<code>i_mem_out</code>	<code>L1MemIf<TRIBE_L2_AXI_WIDTH></code>	Instruction L1 miss/refill path exported to the cluster CDC.

Port	Type	Description
d_mem_out	L1MemIf<TRIBE_L2_AXI_WIDTH>	Data L1, AMO, and shared MMU page-table-walk path exported to the cluster CDC.
clint_msip_in	bool	CLINT machine software interrupt pending input, when interrupts are enabled.
clint_mtip_in	bool	CLINT machine timer interrupt pending input, when interrupts are enabled.
time_lo_in	uint32_t	Low 32 bits of platform time for CSR/time reads, normally driven by CLINT.
time_hi_in	uint32_t	High 32 bits of platform time for CSR/time reads, normally driven by CLINT.
external_irq_in	bool	External interrupt input, normally driven by PLIC when interrupts are enabled.
sbi*_out, sbi*_in	scalar ports	Local timer requests and multicore IPI/FENCE.I/SFENCE.VMA routing.
debug_core_out through debug_decode_out	grouped debug structs	Coherent snapshots of core, MMU, cache, writeback, CSR, IRQ, register, branch, and decode state.
debug_sbi_out	TribeSbiDebug	Grouped SBI request/handling snapshot.
perf_out	TribePerf	Per-cycle performance/stall/cache debug snapshot.
debugen_in	bool	C++ simulation debug print enable flag.

The main combinational control in **Tribe** handles pipeline hazards, branch redirects, global memory wait, MMU page-table-walk arbitration, trap redirection, SBI timer emulation, I-cache/TLB invalidation, and late load forwarding. The DMMU and IMMU page-table walkers share the L2 data-side port with normal data-cache traffic; DMMU requests have priority over IMMU requests after normal data-cache reads/writes. The data MMU has a direct

physical window for the IO region so MMIO is not translated by Sv32.

Tribe itself does not contain L2, DRAM, or memory-mapped devices. It exposes two **L1MemIf** channels plus interrupt, timer, SBI, and invalidation ports. **TribeTest**<CPU_CORES> connects those channels to the shared L2 through CDCs and provides the external AXI boundary used by the simulation and SoC wrappers.

Decode

Ports:

Port	Type	Description
<code>pc_in</code>	<code>uint32_t</code>	PC associated with <code>instr_in</code> .
<code>instr_valid_in</code>	<code>bool</code>	Indicates that <code>instr_in</code> is valid for decode.
<code>instr_in</code>	<code>uint32_t</code>	32-bit instruction word from I-cache; compressed instructions are decoded from low bits.
<code>regs_data0_in</code>	<code>uint32_t</code>	Register-file value for decoded <code>rs1</code> .
<code>regs_data1_in</code>	<code>uint32_t</code>	Register-file value for decoded <code>rs2</code> .
<code>rs1_out</code>	<code>u<5></code>	Decoded source register 1 index for register-file read.
<code>rs2_out</code>	<code>u<5></code>	Decoded source register 2 index for register-file read.
<code>state_out</code>	<code>State</code>	Decoded <code>State</code> carrying operands and control fields into execute.

Decode selects the active decoder specification from **Rv32im**, **Rv32ia**, and **Zicsr** according to build flags. It fills the pipeline **State**, attaches the current PC, marks validity from `instr_valid_in`, fetches register values, and handles the AUIPC PC operand special case. It is combinational and has no internal register state.

Execute

Ports:

Port	Type	Description
state_in	State	Execute-stage State after forwarding and trap redirection.
alu_result_out	uint32_t	ALU result. For comparisons, upper result bits carry equality information used by branches.
debug_alu_a_out	uint32_t	Debug ALU operand A.
debug_alu_b_out	uint32_t	Debug ALU operand B.
branch_taken_out	bool	Branch/jump taken result.
branch_target_out	uint32_t	Resolved branch or jump target.

Execute contains the ALU and branch decision logic. It supports integer arithmetic, shifts, comparisons, multiply/divide operations, address calculation for memory operations, and branch target calculation. Trap and return redirection are represented as branch-like state before this stage enters **Execute**.

ExecuteMem

Ports:

Port	Type	Description
state_in	State	Execute-stage State after forwarding/trap-return adjustments.
alu_result_in	uint32_t	Effective address or ALU result from Execute .
dcache_read_valid_in	bool	Atomic read response valid, when atomics are enabled.
dcache_read_addr_in	uint32_t	Address tag returned with D-cache read data.
dcache_read_expected_addr_in	uint32_t	Expected physical address for atomic read completion.
dcache_read_data_in	uint32_t	D-cache read data used by AMO operations.
mem_stall_in	bool	Holds issued memory request while D-cache/L2 cannot accept it.

Port	Type	Description
hold_in	bool	Holds request metadata while the pipeline waits for writeback.
mem_write_out	bool	Registered store request to D-cache.
mem_write_addr_out	uint32_t	Store address to D-cache.
mem_write_data_out	uint32_t	Store data to D-cache.
mem_write_mask_out	uint8_t	Store byte mask to D-cache.
mem_read_out	bool	Registered load request to D-cache.
mem_read_addr_out	uint32_t	Load address to D-cache.
mem_split_out	bool	Current access crosses a 32-byte L1 line and must be split.
mem_split_busy_out	bool	A delayed second split transaction is pending.
split_load_out	bool	Writeback must assemble a split load from two words.
split_load_low_out	uint32_t	Low aligned address for split-load matching.
split_load_high_out	uint32_t	High aligned address for split-load matching.
atomic_busy_out	bool	Atomic LR/SC/AMO sequence is active, when atomics are enabled.
atomic_sc_result_out	uint32_t	Store-conditional architectural result, 0 for success and 1 for failure.

ExecuteMem turns decoded memory operations into D-cache requests. It detects accesses that cross an L1 cache line and issues the first and second aligned transactions in order, while providing metadata for **WritebackMem** to reconstruct split loads. With **ENABLE_RV32IA**, it also implements LR/SC reservation tracking and AMO read-modify-write sequencing.

Writeback

Ports:

Port	Type	Description
state_in	State	Writeback-stage State .

Port	Type	Description
alu_result_in	uint32_t	ALU result to write for ALU instructions.
mem_data_in	uint32_t	Raw aligned load data.
mem_data_hi_in	uint32_t	High word for legacy split-load interface; normally zero after WritebackMem assembly.
mem_addr_in	uint32_t	Load address used for byte/halfword interpretation.
mem_split_in	bool	Indicates split-load result selection.
regs_data_out	uint32_t	Final architectural value for the integer register file.
regs_wr_id_out	uint8_t	Destination register index.
regs_write_out	bool	Register-file write enable.

Writeback formats the final value for the integer register file. It selects PC+2, PC+4, ALU result, or memory result according to `State::wb_op`, and sign- or zero-extends byte and halfword loads based on `funct3`. Register x0 is still protected by the register file and write-enable logic around this stage.

WritebackMem

Ports:

Port	Type	Description
state_in	State	Writeback-stage State .
alu_result_in	uint32_t	Address expected for the active load response.
split_load_in	bool	Indicates that two aligned read responses must be assembled.
split_load_low_addr_in	uint32_t	Low aligned address of a split load.
split_load_high_addr_in	uint32_t	High aligned address of a split load.
dcache_read_valid_in	bool	D-cache read response valid.
dcache_read_addr_in	uint32_t	D-cache response address tag.
dcache_read_data_in	uint32_t	D-cache read data.

Port	Type	Description
dcache_write_valid_in	bool	Store request accepted/visible to D-cache.
dcache_write_addr_in	uint32_t	Store address for forwarding history.
dcache_write_data_in	uint32_t	Store data for forwarding history.
dcache_write_mask_in	uint8_t	Store byte mask for forwarding history.
hold_in	bool	Keeps pending load information while pipeline is stalled.
load_ready_out	bool	Load result is available for register writeback and forwarding.
load_raw_out	uint32_t	Raw load data before architectural sign/zero extension.
load_result_out	uint32_t	Architecturally extended load value for late forwarding.
wb_mem_data_out	uint32_t	Load data passed to Writeback .
wb_mem_data_hi_out	uint32_t	High split-load word passed to Writeback .

WritebackMem is the one-cycle memory/writeback boundary. It captures D-cache responses, waits until both halves of a split load are available, assembles unaligned words, and performs short store-to-load forwarding. Forwarding is disabled for atomics so AMO semantics are not hidden by local store history.

Peripheral

L1Cache

Ports:

Port	Type	Description
write_in	bool	CPU write request. The data L1 is write-through to L2.

Port	Type	Description
<code>write_data_in</code>	<code>uint32_t</code>	32-bit write data.
<code>write_mask_in</code>	<code>uint8_t</code>	Byte mask for the 32-bit write.
<code>read_in</code>	<code>bool</code>	CPU read request.
<code>addr_in</code>	<code>uint32_t</code>	CPU byte address.
<code>read_data_out</code>	<code>uint32_t</code>	32-bit CPU read result.
<code>read_addr_out</code>	<code>uint32_t</code>	Address tag associated with <code>read_data_out</code> .
<code>read_valid_out</code>	<code>bool</code>	Read response valid.
<code>busy_out</code>	<code>bool</code>	Cache cannot accept or complete the current request.
<code>stall_in</code>	<code>bool</code>	Front-end/pipeline stall input.
<code>flush_in</code>	<code>bool</code>	Redirect flush input, used by I-cache fetch.
<code>invalidate_in</code>	<code>bool</code>	Clears valid tags for FENCE.I/SFENCE.VMA.
<code>invalidate_line_in</code>	<code>bool</code>	Invalidates one addressed set after a peer/DMA store.
<code>invalidate_addr_in</code>	<code>uint32_t</code>	Address selecting the set for targeted invalidation.
<code>cache_disable_in</code>	<code>bool</code>	Forces direct, uncached reads for MMIO/direct paths.
<code>mem_out</code>	<code>L1MemIf<PORT_BITWIDTH></code>	Grouped read/write/refill request, read data, and wait handshake toward L2.
<code>perf_out</code>	<code>L1CachePerf</code>	L1 hit/wait/state performance snapshot.

L1Cache is a small set-associative cache with 32-byte lines. It is instantiated twice: `icache` with ID 0 and `dcache` with ID 1. The line is stored in even/odd 16-bit RAM halves, then assembled into 32-bit words on reads and refills. The data L1 is write-through; it sends 32-bit stores directly to L2 and does not own dirty state.

The implementation is split into geometry, state, request, refill, lookup, response, and controller layers, each with a focused native unit test. The L1 refill port width equals the selected L2 AXI width and may be smaller than the cache line. A miss refills a line over one or more `PORT_BITWIDTH` beats. For data reads that would cross the final word of a cache line, the CPU split logic handles the access before L2 sees a cacheable fill. MMIO/direct reads bypass L1 caching. Cached RAM direct reads stay aligned to the containing L2 beat so dirty data

already present in L2 is used instead of stale backing RAM.

L2Cache

Ports:

Port	Type	Description
i_mem_in	L1MemIf<PORT_BITWIDTH>[CPU_PORTS]	Instruction refill request/response interfaces.
d_mem_in	L1MemIf<PORT_BITWIDTH>[CPU_PORTS]	Data, AMO, and page-table-walk request/response interfaces.
memory_base_in	uint32_t	Base physical address of the memory map.
memory_size_in	uint32_t	Total visible bytes across all regions.
mem_region_size_in	uint32_t[MEM_PORTS]	Per-region byte sizes. Regions are contiguous, not interleaved.
mem_region_uncached_in	bool[MEM_PORTS]	Per-region bypass flag; device/MMIO regions are uncached.
axi_in	Axi4If<ADDR_BITS, 4, PORT_BITWIDTH>[MEM_PORTS]	Coherent full-address slave ports for external AXI masters such as DMA.
axi_out	Axi4If<MEM_ADDR_BITS, 4, PORT_BITWIDTH>[MEM_PORTS]	Master ports to RAM/device regions.
dma_line*_in, dma_line_ready_out	full-line sideband	Optional coherent cache-line allocation path with byte keep mask.

L2Cache is a configurable set-associative shared cache with 32-byte lines and a configurable memory beat width; the default Tribe configuration is four-way and 64 KiB. It arbitrates coherent external AXI slave requests, each core's data request, and each core's instruction request onto one L2 tag/data RAM. External AXI masters can read cached data written by a CPU, and CPUs can read data written through an external AXI slave port.

The request, memory, tag/data, wait, and controller layers operate only on `l2_clock`. A selected request is captured in `CacheRequest`; completion is captured in a per-endpoint `CacheResponse`. CPU wait is released only when the registered response matches the original port, address, and operation, so an L1 request can safely remain asserted across multiple faster `clk` edges. Round-robin CPU selection prevents a continuously active low-index core from starving another core.

L2 memory ports are split into contiguous regions. Address selection subtracts `memory_base_in`, chooses a region by cumulative region size, and then forwards a local byte address to the selected AXI master port. Cached regions allocate and evict dirty cache lines through AXI. Uncached regions bypass the tag/data RAM and issue single-beat MMIO reads and writes, so device regions do not infer cache storage.

BranchPredictor

Ports:

Port	Type	Description
<code>lookup_valid_in</code>	<code>bool</code>	Valid decoded branch lookup.
<code>lookup_pc_in</code>	<code>uint32_t</code>	Branch PC for lookup.
<code>lookup_target_in</code>	<code>uint32_t</code>	Newly decoded branch target.
<code>lookup_fallthrough_in</code>	<code>uint32_t</code>	Sequential next PC.
<code>lookup_br_op_in</code>	<code>u<4></code>	Branch operation type.
<code>predict_taken_out</code>	<code>bool</code>	Predicted branch-taken flag.
<code>predict_next_out</code>	<code>uint32_t</code>	Predicted next PC.
<code>update_valid_in</code>	<code>bool</code>	Execute-stage branch update valid.
<code>update_pc_in</code>	<code>uint32_t</code>	Resolved branch PC.
<code>update_taken_in</code>	<code>bool</code>	Resolved branch direction.
<code>update_target_in</code>	<code>uint32_t</code>	Resolved branch target.

`BranchPredictor` is a direct-mapped predictor with saturating counters, valid bits, tags, and target storage. Conditional branches use the counter state. JAL, JALR, and JR are predicted taken. On execute resolution, the predictor updates direction and target state.

Interrupt Controller

Ports:

Port	Type	Description
<code>mstatus_in</code>	<code>uint32_t</code>	CSR <code>mstatus</code> bits used for global interrupt enables.
<code>mie_in</code>	<code>uint32_t</code>	CSR interrupt enable mask.
<code>mideleg_in</code>	<code>uint32_t</code>	Machine interrupt delegation mask.
<code>mip_sw_in</code>	<code>uint32_t</code>	Software-writable pending bits from CSR state.
<code>priv_in</code>	<code>u<2></code>	Current privilege mode.
<code>clint_msip_in</code>	<code>bool</code>	Machine software interrupt from CLINT.

Port	Type	Description
<code>clint_mtip_in</code>	<code>bool</code>	Machine timer interrupt from CLINT.
<code>external_irq_in</code>	<code>bool</code>	External interrupt request, normally PLIC output.
<code>mip_out</code>	<code>uint32_t</code>	Merged interrupt-pending bits.
<code>interrupt_valid_out</code>	<code>bool</code>	An enabled interrupt should be taken.
<code>interrupt_cause_out</code>	<code>uint32_t</code>	Selected interrupt cause number.
<code>interrupt_to_supervisor_out</code>	<code>bool</code>	Interrupt should trap to supervisor mode by delegation.

`InterruptController` merges hardware CLINT interrupt inputs, external interrupt input, and writable CSR pending bits. It masks pending bits with `mie`, applies privilege-aware global enable rules from `mstatus`, applies `mideleg`, and reports one pending cause to the CSR/trap block. The external interrupt path is used by PLIC for Linux UART interrupts.

MMU/TLB

Ports:

Port	Type	Description
<code>vaddr_in</code>	<code>uint32_t</code>	Virtual address to translate.
<code>read_in</code>	<code>bool</code>	Load translation request.
<code>write_in</code>	<code>bool</code>	Store translation request.
<code>execute_in</code>	<code>bool</code>	Instruction-fetch translation request.
<code>satp_in</code>	<code>uint32_t</code>	Current <code>satp</code> ; Sv32 is active when MODE is 1.
<code>priv_in</code>	<code>u<2></code>	Current privilege mode.
<code>direct_base_in</code>	<code>uint32_t</code>	Base of direct physical bypass window.
<code>direct_size_in</code>	<code>uint32_t</code>	Size of direct physical bypass window.
<code>fill_in</code>	<code>bool</code>	External/manual TLB fill request.
<code>fill_index_in</code>	<code>u<clog2(ENTRIES)></code>	External/manual fill index.
<code>fill_vpn_in</code>	<code>uint32_t</code>	External/manual fill virtual page number.
<code>fill_ppn_in</code>	<code>uint32_t</code>	External/manual fill physical page number.
<code>fill_flags_in</code>	<code>uint8_t</code>	External/manual fill PTE flags.
<code>sfence_in</code>	<code>bool</code>	Invalidates cached translations.
<code>mem_read_out</code>	<code>bool</code>	Page-table-walker memory read request.
<code>mem_addr_out</code>	<code>uint32_t</code>	Physical PTE address requested by the walker.
<code>mem_read_data_in</code>	<code>uint32_t</code>	32-bit PTE data returned by memory.
<code>mem_wait_in</code>	<code>bool</code>	Page-table-walker memory wait.

Port	Type	Description
<code>paddr_out</code>	<code>uint32_t</code>	Translated or bypassed physical address.
<code>translated_out</code>	<code>bool</code>	Translation is active for the current request.
<code>hit_out</code>	<code>bool</code>	TLB hit or translation disabled.
<code>fault_out</code>	<code>bool</code>	Page fault or permission fault.
<code>miss_out</code>	<code>bool</code>	Translation miss requiring a page-table walk.
<code>busy_out</code>	<code>bool</code>	Walker is active or waiting.
<code>debug_last_pte_out</code>	<code>uint32_t</code>	Last PTE captured by the walker.
<code>debug_last_addr_out</code>	<code>uint32_t</code>	Last PTE address captured by the walker.

MMU_TLB implements a small Sv32 TLB with a hardware page-table walker. There are separate instances for instruction fetch and data access. Translation is enabled only when `satp.MODE == 1` and current privilege is not M-mode. A direct mapping window can bypass translation; Tribe uses it for DMMU access to MMIO so device addresses remain physical under an OS.

The walker reads the level-1 PTE from the `satp` root page and, when needed, reads the level-0 PTE. It supports level-1 superpages and level-0 pages, checks valid/write/read combinations, checks A/D/R/W/X/U permissions, and reports instruction/load/store page faults through the main CSR/trap path. `SFENCE.VMA` clears cached translations.

Axi4RegionMux

Ports:

Port	Type	Description
<code>slave_in</code>	<code>Axi4If<ADDR_WIDTH, ID_WIDTH, DATA_WIDTH></code>	AXI slave side connected to an L2 uncached/device region.
<code>masters_out</code>	<code>Axi4If<ADDR_WIDTH, ID_WIDTH, DATA_WIDTH>[N]</code>	AXI master sides connected to devices.
<code>region_base_in</code>	<code>uint32_t[N]</code>	Device-local base address for each region.
<code>region_size_in</code>	<code>uint32_t[N]</code>	Byte size of each device region.

`Axi4RegionMux` routes one AXI device-region port to several memory-mapped device responders. Regions are decoded by local address range. The mux preserves AXI channel IDs and returns read/write responses from the selected device. Tribe uses this mux for the IO region behind the L2 uncached port.

Axi4Ram

Ports:

Port	Type	Description
axi_in	Axi4If<ADDR_WIDTH, ID_WIDTH, DATA_WIDTH>	AXI access to RAM storage.

Axi4Ram is the common testbench and SoC RAM responder. It accepts single-beat AXI reads and writes, stores data at the configured beat width, and supports checkpoint serialization through its `_strobe(FILE*)` path in native C++ tests. The default Tribe executable wrapper uses three RAM instances; the SoC wrapper keeps two DRAMs in **SystemTest** and places the third memory region inside **System**.

SoC

`tribe_cpu/SoC/System.cpp` defines a **System** DUT and a **SystemTest** wrapper. **System** integrates **TribeTest**<1> with the on-chip IO space, NS16550A UART, CLINT, PLIC, Accelerator, SD controller, and the third internal AXI RAM region. **SystemTest** provides external `dram0` and `dram1` RAMs and converts realistic hardware configuration values into **System** input ports before running the same program modes as the corresponding Tribe targets. The full native Linux test wrapper in `TribeTest.h` additionally connects the ethgig network device chain and optional TAP socket bridge.

System ports:

Port	Type	Description
reset_pc_in	uint32_t	Reset PC forwarded to Tribe .
boot_hartid_in	uint32_t	Boot hart id forwarded as initial <code>a0</code> .
boot_dtb_addr_in	uint32_t	Boot DTB address forwarded as initial <code>a1</code> .
boot_priv_in	u<2>	Initial privilege mode.
memory_base_in	uint32_t	Physical base of the memory map.
memory_size_in	uint32_t	Total visible memory-map size.
mem_region_size_in	uint32_t[L2_MEM_PORTS]	Per-region byte sizes forwarded to L2.
uart_rx_valid_in	bool	External UART RX byte valid.
uart_rx_data_in	uint8_t	External UART RX byte.

Port	Type	Description
uart_rx_ready_out	bool	UART can accept another RX byte.
uart_tx_valid_out	bool	UART transmitted a byte this cycle.
uart_tx_data_out	uint8_t	UART transmitted byte.
sd_cmd_valid_out	bool	SD physical command/data byte stream valid.
sd_cmd_data_out	u<8>	SD physical command/data byte.
sd_cmd_last_out	bool	Last byte of the current SD command/data frame.
sd_cmd_ready_in	bool	External SD card model can accept another command/data byte.
sd_rsp_valid_in	bool	External SD card model response byte valid.
sd_rsp_data_in	u<8>	External SD card model response byte.
sd_rsp_last_in	bool	Last response byte from SD card model.
sd_rsp_ready_out	bool	SD controller can accept another response byte.
perf_out	TribePerf	CPU performance snapshot.
dmem_write_out	bool	CPU debug data-memory write indication.
dmem_write_data_out	uint32_t	CPU debug data-memory write data.
dmem_write_mask_out	uint8_t	CPU debug data-memory write mask.
dmem_read_out	bool	CPU debug data-memory read indication.
dmem_addr_out	uint32_t	CPU debug data-memory address.
imem_read_addr_out	uint32_t	CPU debug instruction-memory address.

Port	Type	Description
debug_core_out, debug_mmu_out, debug_cache_out, debug_wb_out, debug_csr_out, debug_regs_out, debug_branch_out, debug_decode_out	grouped debug structs	Optional coherent debug snapshots forwarded from core 0.
axi_out	Axi4If<clog2(MAX_RAM_SIZE), 4, TRIBE_L2_AXI_WIDTH>[2]	External AXI master ports for dram0 and dram1 in SystemTest.

The SoC memory map mirrors the default Tribe executable wrapper for CPU-visible RAM: two external DRAM regions, one internal RAM region, and one uncached IO region. Inside System IO, Axi4RegionMux maps UART at offset 0x0, CLINT at offset 0x100, Accelerator at offset 0xC100, SD controller at offset 0xD100, and PLIC at offset 0x10000. PLIC source 1 is driven by the NS16550A UART IRQ and source 2 is driven by the SD controller IRQ. The native TestTribe wrapper uses a six-device IO mux and adds ethgig at offset 0xE000; its PLIC also carries Ethernet interrupt sources.

Devices

Device modules live in `tribe_cpu/devices`. They are memory-mapped AXI responders; devices with DMA (Accelerator, SDController, and EthGigDMA) also own AXI master ports that connect to L2 coherent slave ports. In the default Tribe simulation wrapper and in `System`, devices are placed behind an IO-space region mux connected to the uncached L2 IO region.

IOUART

Ports:

Port	Type	Description
axi_in	Axi4If<ADDR_WIDTH, ID_WIDTH, DATA_WIDTH>	MMIO register access.
uart_valid_out	bool	One-cycle pulse when a byte is written.
uart_data_out	uint8_t	Byte written by software.

IOUART is a minimal UART-like output device. It exposes TXDATA at offset 0x00 and STATUS at offset 0x04; status bit 0 is always ready. Writes to TXDATA emit `uart_valid_out` and `uart_data_out`. It is useful for simple bare-metal tests.

NS16550A

Ports:

Port	Type	Description
<code>axi_in</code>	<code>Axi4If<ADDR_WIDTH, ID_WIDTH, DATA_WIDTH></code>	MMIO register access to the 16550-style register file.
<code>uart_valid_out</code>	<code>bool</code>	One-cycle pulse when software writes the transmit holding register.
<code>uart_data_out</code>	<code>uint8_t</code>	Transmitted byte.
<code>uart_rx_valid_in</code>	<code>bool</code>	External RX byte valid.
<code>uart_rx_data_in</code>	<code>uint8_t</code>	External RX byte.
<code>uart_rx_ready_out</code>	<code>bool</code>	Single-byte RX buffer is empty and can accept a byte.
<code>irq_out</code>	<code>bool</code>	UART interrupt request, used as PLIC source 1 in the wrappers.

NS16550A models enough of a 16550-compatible UART for firmware and Linux console probing. It implements the usual register offsets for RBR/THR/DLL, IER/DLM, IIR/FCR, LCR, MCR, LSR, MSR, and SCR. Transmit is modeled as always empty and ready by setting `LSR.THRE` and `LSR.TEMT`; THR-empty interrupts are intentionally not generated, so Linux can use polling for TX without interrupt storms. Receive uses a one-byte buffer, sets `LSR.DR`, returns `IIR=0x04` when RX interrupt enable is set, clears RBR on read, and asserts `irq_out` when `MCR.OUT2` and `IER.RDI` are enabled. DLAB selects divisor latch registers at offsets 0 and 1.

CLINT

Ports:

Port	Type	Description
<code>axi_in</code>	<code>Axi4If<ADDR_WIDTH, ID_WIDTH, DATA_WIDTH></code>	MMIO access to CLINT registers.
<code>set_mtimecmp_in</code>	<code>bool</code>	Direct timer-compare update from local SBI <code>set_timer</code> emulation.
<code>set_mtimecmp_lo_in</code>	<code>uint32_t</code>	Low 32 bits for direct <code>mtimecmp</code> update.
<code>set_mtimecmp_hi_in</code>	<code>uint32_t</code>	High 32 bits for direct <code>mtimecmp</code> update.

Port	Type	Description
<code>msip_out</code>	<code>bool</code>	Machine software interrupt pending.
<code>mtip_out</code>	<code>bool</code>	Machine timer interrupt pending.
<code>set_mtimecmp_per_hart_in</code>	arrays	Multicore SBI timer updates, one compare value per hart.
<code>msip_per_hart_out</code> , <code>mtip_per_hart_out</code>	<code>bool[HART_COUNT]</code>	Multicore software/timer interrupt lines.
<code>debug_mtime_lo_out</code>	<code>uint32_t</code>	Low 32 bits of current <code>mtime</code> , also forwarded to <code>Tribe.time_lo_in</code> .
<code>debug_mtime_hi_out</code>	<code>uint32_t</code>	High 32 bits of current <code>mtime</code> , also forwarded to <code>Tribe.time_hi_in</code> .
<code>debug_mtimecmp_lo_out</code>	<code>uint32_t</code>	Low 32 bits of current <code>mtimecmp</code> .
<code>debug_mtimecmp_hi_out</code>	<code>uint32_t</code>	High 32 bits of current <code>mtimecmp</code> .

CLINT implements the basic RISC-V local interrupt timer registers used by Tribe: `msip`, per-hart `mtimecmp`, and shared `mtime`. The timer increments according to `TRIBE_CLINT_TICK_DIV_CONFIG`, which lets Linux runs slow the timer relative to CPU model cycles. In single-core builds, `mtip_out` and `msip_out` expose hart 0. `MULTICORE` builds also expose arrays for all `CPUS_PER_L2_CACHE` harts. The direct `set_mtimecmp_*` inputs let the core emulate legacy SBI timer calls without requiring firmware support.

PLIC

Ports:

Port	Type	Description
<code>axi_in</code>	<code>Axi4If<ADDR_WIDTH, ID_WIDTH, DATA_WIDTH></code>	MMIO access to PLIC priority, pending, enable, threshold, and claim/complete registers.
<code>source_irq_in</code>	<code>bool[SOURCES]</code>	Level-sensitive interrupt source inputs. Source 0 is unused by convention.
<code>external_irq_out</code>	<code>bool</code>	External interrupt line to the CPU interrupt controller.

Port	Type	Description
<code>external_irq_per_hart_outbool[HART_COUNT]</code>		Multicore external interrupt line for each PLIC context.

PLIC is a compact platform interrupt controller with one context per configured hart. It implements source priority registers, a shared pending register, per-context enable and threshold registers, and claim/complete registers at standard PLIC-style offsets. Device source levels are latched by gateway state so a request remains claimable until a CPU reads the claim register. Writing the claimed source ID to the complete register releases the gateway, allowing a still-asserted device level to pend again later. In the current wrappers, source 1 is connected to `NS16550A.irq_out`, source 2 is connected to `SDController.irq_out`, and native `TestTribe` also connects Ethernet IRQ sources. `external_irq_out` is the hart-0 compatibility output; multicore wrappers use `external_irq_per_hart_out`.

Accelerator

Ports:

Port	Type	Description
<code>axi_in</code>	<code>Axi4If<ADDR_WIDTH, ID_WIDTH, DATA_WIDTH></code>	CPU MMIO control/status and accelerator-local memory access.
<code>dma_out</code>	<code>Axi4If<ADDR_WIDTH, ID_WIDTH, DATA_WIDTH></code>	DMA memory access through an L2 coherent slave port.

Accelerator is a test device with a small local word memory, a PRBS generator, and a DMA engine. Its control registers include source address, destination address, transfer length, control, status, and PRBS seed. The DMA path is a real AXI master port and is intended to connect to an L2 slave/coherency port, not to a private CPU shortcut. It can copy main memory into accelerator memory, copy accelerator memory back to main memory, and fill its local memory from a PRBS sequence for software-visible data movement tests.

SDController

Ports:

Port	Type	Description
<code>axi_in</code>	<code>Axi4If<ADDR_WIDTH, ID_WIDTH, DATA_WIDTH></code>	CPU MMIO register access.

Port	Type	Description
<code>dma_out</code>	<code>Axi4If<ADDR_WIDTH, ID_WIDTH, DATA_WIDTH></code>	DMA master port connected to an L2 coherent slave port.
<code>sd_cmd_valid_out</code>	<code>bool</code>	Command/data byte stream valid toward the SD physical/card model.
<code>sd_cmd_data_out</code>	<code>u<8></code>	Command/data byte toward the SD physical/card model.
<code>sd_cmd_last_out</code>	<code>bool</code>	Last byte of the current command/data frame.
<code>sd_cmd_ready_in</code>	<code>bool</code>	SD physical/card model can accept the next byte.
<code>sd_rsp_valid_in</code>	<code>bool</code>	Response/data byte from SD physical/card model is valid.
<code>sd_rsp_data_in</code>	<code>u<8></code>	Response/data byte from SD physical/card model.
<code>sd_rsp_last_in</code>	<code>bool</code>	Last byte of the response/data frame.
<code>sd_rsp_ready_out</code>	<code>bool</code>	Controller can accept another response byte.
<code>irq_out</code>	<code>bool</code>	Done interrupt when enabled and pending.
<code>dma_write_complete_out</code>	<code>bool</code>	Pulse/debug indication that a DMA write-to-card path completed.
<code>debug_status_out</code>	<code>uint32_t</code>	Current synthesized SD status register.
<code>debug_state_out</code>	<code>uint32_t</code>	Current SD controller state.
<code>debug_count_out</code>	<code>uint32_t</code>	Current byte counter.
<code>debug_len_out</code>	<code>uint32_t</code>	Current requested transfer length.

`SDController` implements a byte-stream SD command/data front end and a CPU-visible MMIO/DMA control block. The implemented card commands are single-block read (`CMD17`) and single-block write (`CMD24`) using the local simple frame format in `SDTypes.h`. Software programs command, argument, length, DMA address, optional descriptor entries, and control bits through registers at offsets `0x00` through `0x34`.

PIO mode moves payload bytes through `TXDATA` and `RXDATA`. DMA mode uses `dma_out` to read or write main memory through L2. The controller supports direct `DMA_ADDR/LEN` trans-

fers and a FIFO of page/list descriptors via `DMA_DESC_ADDR`, `DMA_DESC_LEN`, `DMA_DESC_PUSH`, and `DMA_DESC_STATUS`. Status bits report busy, done, error, RX valid, TX ready, IRQ pending, and descriptor readiness. In the wrappers, the SD IRQ is PLIC source 2 and DMA completion can request an external cache invalidation so CPU reads see data written by the DMA engine.

SDPhysical, SDFifo, and SDCardVerif

`SDPhysical` and `SDFifo` are small SD-layer helpers under `tribe_cpu/devices/sd`. `SDPhysical` handles the byte-level command/response stream between controller logic and card-facing ports, while `SDFifo` provides small byte FIFOs used by SD tests and layering. `SDTypes.h` is the shared register, status, control, IRQ, and command constant header.

`tribe_cpu/verif/SDCardVerif.h` provides a C++ SD card verification model plus `SDCardVerifFrontend`, an RTL-facing wrapper that connects to `SDController` physical ports. The verification model stores a byte vector, can load/save an SD image file, supports checkpointing, and responds to the controller's `CMD17/CMD24` frames. The native Linux wrapper uses this model for `TRIBE_LINUX_SD_IMAGE` and can override or restore SD image state across checkpoints.

EthGigDMA

Ports:

Port	Type	Description
<code>axi_in</code>	<code>Axi4If<ADDR_WIDTH, ID_WIDTH, DATA_WIDTH></code>	CPU MMIO access to Xilinx AXI Ethernet MAC and AXI DMA compatible registers.
<code>dma_out</code>	<code>Axi4If<ADDR_WIDTH, ID_WIDTH, DATA_WIDTH></code>	DMA master port connected to an L2 coherent slave port.
<code>mac_tx_valid_out</code>	<code>bool</code>	TX payload byte valid toward <code>EthGigMAC</code> .
<code>mac_tx_data_out</code>	<code>u<8></code>	TX payload byte toward <code>EthGigMAC</code> .
<code>mac_tx_last_out</code>	<code>bool</code>	Last TX payload byte.
<code>mac_tx_ready_in</code>	<code>bool</code>	MAC can accept another TX byte.
<code>mac_rx_valid_in</code>	<code>bool</code>	RX payload byte from <code>EthGigMAC</code> is valid.
<code>mac_rx_data_in</code>	<code>u<8></code>	RX payload byte from <code>EthGigMAC</code> .
<code>mac_rx_last_in</code>	<code>bool</code>	Last RX payload byte.

Port	Type	Description
mac_rx_ready_out	bool	DMA can accept another RX byte for the current descriptor.
tx_irq_out	bool	TX IOC interrupt after enabled descriptor completion.
rx_irq_out	bool	RX IOC interrupt after enabled descriptor completion.
debug_state_out	uint32_t	Current DMA state.
debug_tx_sr_out	uint32_t	Current TX DMA status register.
debug_rx_sr_out	uint32_t	Current RX DMA status register.
local_mac_out	logic<48>	MAC address programmed through Xilinx AXI Ethernet address registers.
promisc_out	bool	Promiscuous receive mode from the frame match register.

EthGigDMA is the CPU-facing network device block. Its register map is intentionally compatible with the Linux Xilinx AXI Ethernet driver family: AXI DMA TX/RX control, status, current descriptor, and tail descriptor registers are at the standard DMA offsets, while the MAC register window implements the AXI Ethernet RAF, interrupt, address, receive/transmit control, flow control, EMMC/PHYC, MDIO, and filter registers used by the driver. The hardware supports descriptor-based TX and RX, reads TX descriptors and payload through `dma_out`, streams TX bytes into the MAC, writes RX payloads into descriptor buffers, and updates descriptor status/app words before raising IOC interrupts.

EthGigMAC

Ports:

Port	Type	Description
local_mac_in	logic<48>	Local MAC address used for RX filtering.
local_ip_in	uint32_t	Local IPv4 address used with <code>local_mask_in</code> for optional RX filtering.

Port	Type	Description
local_mask_in	uint32_t	IPv4 subnet mask for RX filtering; zero disables the IP check.
promisc_in	bool	Accept all destination MAC addresses.
tx_valid_in	bool	TX payload byte from DMA is valid.
tx_data_in	u<8>	TX payload byte from DMA.
tx_last_in	bool	Last TX payload byte from DMA.
tx_ready_out	bool	MAC TX FIFO can accept another payload byte.
rx_valid_out	bool	RX payload byte toward DMA is valid.
rx_data_out	u<8>	RX payload byte toward DMA.
rx_last_out	bool	Last RX payload byte toward DMA.
rx_ready_in	bool	DMA can accept another RX payload byte.
pcs_tx_valid_out	bool	Framed TX byte toward PCS is valid.
pcs_tx_data_out	u<8>	Framed TX byte toward PCS.
pcs_tx_last_out	bool	Last framed TX byte toward PCS.
pcs_tx_ready_in	bool	PCS can accept another framed TX byte.
pcs_rx_valid_in	bool	Framed RX byte from PCS is valid.
pcs_rx_data_in	u<8>	Framed RX byte from PCS.
pcs_rx_last_in	bool	Last framed RX byte from PCS.
pcs_rx_ready_out	bool	MAC can accept another framed RX byte.
tx_frames_out	uint32_t	Transmitted frame counter.
rx_frames_out	uint32_t	Received frame counter.
tx_bytes_out	uint32_t	Transmitted payload byte counter.
rx_bytes_out	uint32_t	Received payload byte counter.

EthGigMAC builds and checks Ethernet framing around DMA payload streams. TX inserts seven preamble bytes, SFD, payload, minimum-frame padding to 60 payload bytes, CRC/FCS, and inter-packet gap. RX seeks preamble/SFD, collects a frame, checks the Ethernet CRC residue, filters by broadcast/local MAC/promiscuous mode and optional IPv4 subnet, strips FCS, and emits only payload bytes to DMA.

EthGigPCS

Ports:

Port	Type	Description
tx_valid_in	bool	TX byte from MAC is valid.
tx_data_in	u<8>	TX byte from MAC.
tx_last_in	bool	Last TX byte from MAC.
tx_ready_out	bool	PCS TX FIFO can accept another byte.
tx_valid_out	bool	TX byte toward PHY is valid.
tx_data_out	u<8>	TX byte toward PHY.
tx_last_out	bool	Last TX byte toward PHY.
tx_ready_in	bool	PHY can accept another TX byte.
rx_valid_in	bool	RX byte from PHY is valid.
rx_data_in	u<8>	RX byte from PHY.
rx_last_in	bool	Last RX byte from PHY.
rx_ready_out	bool	PCS RX FIFO can accept another byte.
rx_valid_out	bool	RX byte toward MAC is valid.
rx_data_out	u<8>	RX byte toward MAC.
rx_last_out	bool	Last RX byte toward MAC.
rx_ready_in	bool	MAC can accept another RX byte.

EthGigPCS is currently a buffering layer rather than a full 8b/10b PCS. It preserves byte/last streams in both directions and provides backpressure decoupling between MAC and PHY.

EthGigPHY

Ports:

Port	Type	Description
tx_valid_in	bool	TX byte from PCS is valid.
tx_data_in	u<8>	TX byte from PCS.
tx_last_in	bool	Last TX byte from PCS.
tx_ready_out	bool	PHY can accept another TX byte.
rx_valid_out	bool	RX byte toward PCS is valid.
rx_data_out	u<8>	RX byte toward PCS.

Port	Type	Description
<code>rx_last_out</code>	bool	Last RX byte toward PCS.
<code>rx_ready_in</code>	bool	PCS can accept another RX byte.
<code>rgmii_tx_ctl_out</code>	bool	TX nibble control/valid toward RGMII media.
<code>rgmii_txd_out</code>	u<4>	TX RGMII nibble.
<code>rgmii_tx_last_out</code>	bool	Last TX nibble of the current frame.
<code>rgmii_rx_ctl_in</code>	bool	RX nibble control/valid from RGMII media.
<code>rgmii_rxd_in</code>	u<4>	RX RGMII nibble.
<code>rgmii_rx_last_in</code>	bool	Last RX nibble of the current frame.
<code>mdio_mdc_in</code>	bool	MDIO management clock from host MAC side.
<code>mdio_host_oe_in</code>	bool	Host drives MDIO data when asserted.
<code>mdio_host_data_in</code>	bool	Host-driven MDIO data bit.
<code>mdio_data_out</code>	bool	PHY MDIO data output or pull-up value.
<code>mdio_drive_out</code>	bool	PHY is actively driving MDIO data.

EthGigPHY converts bytes into low/high RGMII nibbles and reconstructs RX bytes from nibbles. It also implements a small MDIO register file and management state machine sufficient for driver probing and fixed 1G link behavior in simulation.

RGMIIVerif and ethgig_tap

`tribe_cpu/verif/RGMIIVerif.h` contains a C++ packet-level RGMII verification model and `RGMIIVerifFrontend`, an RTL-facing module with RGMII nibble ports. Tests can push RX packets into the model and pop packets transmitted by the DUT. The native Linux wrapper can also connect the RGMII verification link to a host TAP process through `TRIBE_LINUX_ETH_TAP_SOCKET`.

`tribe_cpu/linux/net/ethgig_tap.cpp` is the host-side bridge. It creates or uses a TAP interface, exchanges packet frames over a Unix-domain socket, and lets Linux running inside Tribe communicate with the host network namespace as `eth0`. The helper script under `tribe_cpu/linux/net` configures the TAP side; the simulator side connects by passing `TRIBE_LINUX_ETH_TAP_SOCKET=/tmp/tribe-ethgig.sock` to `run_linux_probe.sh`.

Tests

All tests are registered when CMake is configured with `BUILD_TESTING=ON` (the default when the top-level project includes CTest). CTest names are stable; their numeric IDs are not. Tests that need a RISC-V compiler return CTest skip code 77 when `riscv32-unknown-elf-gcc` and `g++` are unavailable. Set `RISCV_HOME` to the toolchain root or place the tools in `PATH`; the bare-metal build helper additionally probes `RISCV` and `$HOME/riscv`.

Decode And Cache Layer Tests

CTest	Coverage
<code>tribe_spec_decode_tests</code>	Generated opcode corpus against RV32I/C/M/A and Zicsr decode semantics.
<code>cache_l1_L1CacheGeometry_test</code>	Set, tag, line, beat, and cacheability calculations.
<code>cache_l1_L1CacheState_test</code>	L1 state records, defaults, and storage ownership.
<code>cache_l1_L1CacheRequest_test</code>	Input capture and grouped memory-driver construction.
<code>cache_l1_L1CacheRefill_test</code>	Multi-beat refill accumulation and word assembly.
<code>cache_l1_L1CacheLookup_test</code>	Associative tag lookup and selected-way data.
<code>cache_l1_L1CacheResponse_test</code>	Held responses, busy, and performance output grouping.
<code>cache_l1_L1CacheCoherence_test</code>	Full and targeted peer invalidation behavior.
<code>cache_l1_L1Cache_test</code>	Integrated layered L1 controller behavior.
<code>cache_l2_L2CacheGeometry_test</code>	L2 address decomposition and crossing predicates.
<code>cache_l2_L2CacheByteOps_test</code>	Masked/unaligned store and read helpers.
<code>cache_l2_L2CacheRegionRouter_test</code>	Contiguous memory-region routing and uncached flags.
<code>cache_l2_L2CacheTagOps_test</code>	Tag packing, dirty/valid extraction, and hit search.
<code>cache_l2_L2CacheResponseGlue_test</code>	Beat slicing and cross-line read assembly.
<code>cache_l2_L2CacheTimeoutOps_test</code>	Progress-age and timeout predicates.
<code>cache_l2_L2CachePortOps_test</code>	Request priority and CPU wait policy.
<code>cache_l2_L2CachePipeline_test</code>	Registered request/response pipeline records.
<code>cache_l2_L2Cache_test</code>	Integrated layered L2 controller behavior.

These focused layer tests are native C++ unit tests. End-to-end `L1Cache_test` and `L2Cache_test` below additionally exercise generated RTL and Verilator.

Module And Platform Tests

CTest base name	Main coverage	Backends
Accelerator_test	Direct accelerator MMIO/DMA plus bare-metal CPU integration.	C++ and _verilator
BranchPredictor_test	Counter updates, targets, prediction, and alias behavior.	C++ and _verilator
CLINT_test	Timer, software interrupt, SBI timer path, and CPU integration.	C++ and _verilator
CPU_test	Fence, CSR/time, byte/unaligned copies, traps, SBI, IRQ hazards, and atomics.	C++ and _verilator
Checkpoint_test	Save/restore of CPU, cache, CDC, memory, UART, CLINT, and PLIC state.	C++ and _verilator
Cpp_test	Bare-metal C++ runtime behavior on the CPU.	C++ and _verilator
EthGigCPU_test	CPU-visible Ethernet registers, descriptors, DMA, and cache invalidation.	C++ and _verilator
EthGigDMA_test	Direct Ethernet DMA descriptor and stream engine behavior at multiple widths.	C++
EthGigMac_test	Ethernet TX/RX framing, CRC, padding, filtering, PCS/PHY stream path.	C++ and _verilator
ExecuteMem_test	Split memory request and atomic sequencing in isolation.	C++
IOUART_test	Minimal UART direct behavior and CPU software output.	C++ and _verilator
L1Cache_test	Integrated L1 hits, misses, refill widths, writes, bypass, and invalidation.	C++ and _verilator

CTest base name	Main coverage	Backends
L2Cache_test	One CPU I/D pair, divided clock, AXI coherence, refill, eviction, MMIO, and crossings.	C++ and _verilator
L2CacheMulti_test	Same L2 suite with CPUS_PER_L2_CACHE I/D pairs and arbitration.	C++ and _verilator
Linux_test	Linux image/DTB/initramfs preparation and simulator build smoke.	C++ prepare path and Verilator build-only smoke
MMU_TLB_Test	Direct Sv32 translation/walk/fault tests and CPU software integration.	C++ and _verilator
Multicore_test	Four-core boot, peer invalidation, barriers, SBI routing, and AMO ownership.	C++ and _verilator
NS16550A_test	UART register semantics, RX/TX, interrupt behavior, and CPU integration.	C++ and _verilator
PLIC_test	Priority, pending, enable, threshold, gateway, and claim/complete behavior.	C++ and _verilator
Perf_test	Linux-like workload correctness plus performance-regression limits.	C++ and _verilator, serial
SD_test	SD command/data stream, PIO, DMA, descriptors, IRQ, and card model.	C++ and _verilator

ISA And Width Matrices

The same software suites run against `tribe64`, `tribe128`, and `tribe256`, and against `SoC64`, `SoC128`, and `SoC256`. Every family has native and Verilator variants:

CTest pattern	Purpose
Tribe{64,128,256}_rv32_spike_fragments[_verilator]	Simulator-generated instruction fragments on the full Tribe platform.
Tribe{64,128,256}_riscv_tests_rv32[_verilator]	Upstream riscv-tests RV32UI/UM/UA/UC programs.
Tribe{64,128,256}_riscv_arch_test[_verilator]	RISC-V architectural test framework.
Tribe{64,128,256}_riscv_dv[_verilator]	riscv-dv generated RV32IMAC/Zicsr/Zifencei programs.
SoC{64,128,256}_rv32_spike_fragments[_refirator]	Reference fragments through the System wrapper.
SoC{64,128,256}_riscv_tests_rv32[_verilator]	Upstream ISA tests through the System wrapper.
SoC{64,128,256}_riscv_arch_test[_verilator]	Architectural tests through the System wrapper.
SoC{64,128,256}_riscv_dv[_verilator]	Generated instruction tests through the System wrapper.

The converter/build targets are `tribe64`, `tribe128`, `tribe256`, their `*_multicore` variants, and `SoC64`, `SoC128`, `SoC256`. Every generated Tribe/System model is converted with both declared clocks and the correct L2 bus width.

Running Tests

```

1  # List exactly what the current build registered.
2  ctest --test-dir build -N
3
4  # Focused cache-layer tests.
5  ctest --test-dir build --output-on-failure -R '^cache_l[12]_'
6
7  # Native and Verilator module/platform regressions.
8  ctest --test-dir build --output-on-failure \
9      -R '^(CPU|L1Cache|L2Cache|L2CacheMulti|Multicore|CLINT|PLIC|SD|EthGig.*)_test'
10
11 # All project tests, including Tribe and converter regressions.
12 ctest --test-dir build --output-on-failure

```

`tribe_cpu/linux/run_linux_probe.sh` is the manual long-running Linux entry point. It supports single-core or `--multicore` execution and `--bus_width` 64, 128, or 256; it is intentionally separate from the bounded CTest Linux build smoke.